FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

Computer System and Architecture

Machine Problem

FINAL PROJECT

RICOHERMOSO, RAYMOND JR. C.	DOC. HADJI TEJUCO
Name of Student	Name of Professor
NOVEMBER 6, 2026	NOVEMBER 12, 2026
Data Performed	Date Submitted

List the steps that you perform to come up with the program.

Paste the source code and explain the functionalities and take a screenshot of the final output.

DESCRIPTION:

Most simple assembly projects are calculators or menus. They work, but they do not show how a program can remember something between runs, or how a few simple rules can make a program feel alive. The program takes input from the player and shows output on screen, presents a menu with ASCII art, uses procedures and macros to keep the code modular, and reads and writes a file so that the game state is not lost when the program closes.

My program named Pixel Pet is a console game for Linux. The player adopts a pet, gives it a name, and takes care of it using a menu. The pet has three stats, each from 0 to 10, and its ASCII face changes depending on how well it is doing. The pet is saved to a file after every action.

The pet has a name typed by the player, stored and reloaded from the file. The pet's face changes (happy, okay, sad) based on its lowest stat. Stats are shown as progress bars, for example [#####-----]5/10. Every action is auto saved, so even closing the terminal early does not lose progress.

MENU OPTIONS - Every stat is limited to the range 0 to 10. A new pet starts with 5 in all three stats.

Option	Action	Effect on the pet
1	Feed	Fullness +3, Mood +1. Refuses if Fullness is already 10
2	Play	Mood +3, Energy -2, Fullness -1. Refuses if Energy is below 2
3	Sleep	Energy +5, Fullness -1
4	Check Status	Shows the name, the face, and the three stat bars
5	Save and Exit	Saves the pet and ends the program

The program finds the **lowest** of the three stats and uses it to pick a face.

Lowest stat	Face	Message
0 to 2	`( T.T )`	Your pet is SAD. Take care of it!
3 to 5	`( -.- )`	Your pet is doing OKAY.
6 to 10	`( ^o^ )`	Your pet is HAPPY!

The whole pet is stored in one file, `pet.dat`, exactly **19 bytes** long.

Bytes	Content
1 to 16	Pet name (text, padded with zeros)
17	Fullness (0 to 10)
18	Mood (0 to 10)
19	Energy (0 to 10)

Because the name and the three stats are declared one after another in memory, the program can save or load the whole pet with a **single** `sys_write` or `sys_read` of 19 bytes.

COMPILATION:

```
PS C:\Users\Ram2\Documents\vsc csa\hjtnasm> docker run --rm -it -v "${PWD}:/work" -w /work/FINALS hjtnasm bash
>>
root@267c12ec9586:/work/FINALS# nasm -f elf32 finalsproj.asm -o finalsproj.o
ld -m elf_i386 finalsproj.o -o finalsproj
./finalsproj
```

FULL SOURCE CODE:

```
;  PIXEL PET by Raymond Ricohermoso Jr.

PET_SIZE equ 19            ; bytes stored in pet.dat

; ---------------- MACROS ----------------
; print <address>, <length>  -> writes a string to the screen
%macro print 2
    pushad
    mov eax, 4             ; sys_write
```

```
    mov ebx, 1              ; stdout
    mov ecx, %1
    mov edx, %2
    int 0x80
    popad
%endmacro

; raise <stat>, <amount>  -> increase a stat (max 10)
%macro raise 2
    mov ebx, %1
    mov ecx, %2
    call inc_stat
%endmacro

; lower <stat>, <amount>  -> decrease a stat (min 0)
%macro lower 2
    mov ebx, %1
    mov ecx, %2
    call dec_stat
%endmacro

; close_file -> closes the file whose descriptor is in [fd]
%macro close_file 0
    mov eax, 6              ; sys_close
    mov ebx, [fd]
    int 0x80
%endmacro

; exit_program -> terminates the program
%macro exit_program 0
    mov eax, 1              ; sys_exit
    xor ebx, ebx
    int 0x80
%endmacro

; ---------------- DATA ----------------
section .data
banner  db 10
        db " ____  _____ _____ ", 10
        db "|  _ \| ____|_   _|", 10
        db "| |_) |  _|   | |  ", 10
        db "|  __/| |___  | |  ", 10
        db "|_|   |_____| |_|  ", 10
```

```
        db "===== PIXEL PET - VIRTUAL PET =====", 10
banner_len equ $ - banner

menu    db 10
        db "+---------------------+", 10
        db "|  [1] Feed           |", 10
        db "|  [2] Play           |", 10
        db "|  [3] Sleep          |", 10
        db "|  [4] Check Status   |", 10
        db "|  [5] Save and Exit  |", 10
        db "+---------------------+", 10
        db "Choose an option: "
menu_len equ $ - menu

face_happy db 10, "  /\_/\", 10, " ( ^o^ )", 10, "  > ^ <", 10
          db "Your pet is HAPPY!", 10
face_happy_len equ $ - face_happy
face_meh   db 10, "  /\_/\", 10, " ( -.- )", 10, "  > - <", 10
          db "Your pet is doing OKAY.", 10
face_meh_len equ $ - face_meh
face_sad   db 10, "  /\_/\", 10, " ( T.T )", 10, "  > n <", 10
          db "Your pet is SAD. Take care of it!", 10
face_sad_len equ $ - face_sad

msg_adopt   db 10, "A pet egg hatches! Name your pet (max 15 letters): "
msg_adopt_len equ $ - msg_adopt
msg_hello   db "Say hello to "
msg_hello_len equ $ - msg_hello
msg_back    db "Welcome back! Your pet is "
msg_back_len equ $ - msg_back
msg_excl    db "!", 10
msg_excl_len equ $ - msg_excl
msg_feed    db " munches the food. Yum!", 10
msg_feed_len equ $ - msg_feed
msg_full    db " is too full to eat.", 10
msg_full_len equ $ - msg_full
msg_play    db " plays happily!", 10
msg_play_len equ $ - msg_play
msg_tired   db " is too tired to play. Let it sleep!", 10
msg_tired_len equ $ - msg_tired
msg_sleep   db " sleeps... Zzz. Energy restored!", 10
msg_sleep_len equ $ - msg_sleep
msg_card    db 10, "--- Status card of "
```

```
msg_card_len equ $ - msg_card
msg_card2   db " ---", 10
msg_card2_len equ $ - msg_card2
msg_badopt  db "Invalid option. Please choose 1-5.", 10
msg_badopt_len equ $ - msg_badopt
msg_bye     db "Pet saved. See you soon!", 10
msg_bye_len equ $ - msg_bye

lbl_full    db "Fullness: "
lbl_mood    db "Mood    : "
lbl_energy  db "Energy  : "
lbl_len     equ 10
barbuf      db "[----------]"
slash10     db "/10", 10
nl          db 10

def_name    db "Pixel", 0
pet_file    db "pet.dat", 0

; --- the pet record (saved to / loaded from pet.dat, 19 bytes) ---
pet_name    times 16 db 0
pet_full    db 5
pet_mood    db 5
pet_energy  db 5

has_pet     dd 0            ; 1 = pet was loaded from the file

; ---------------- BSS ----------------
section .bss
inbuf   resb 32             ; keyboard input
numbuf  resb 12             ; number -> text conversion
fd      resd 1              ; file descriptor

; ---------------- CODE ----------------
section .text
    global _start

_start:
    cld
    call load_pet           ; READ pet.dat (if it exists)
    print banner, banner_len
    cmp dword [has_pet], 0
    jne .welcome_back
```

```
    ; --- new pet: ask for a name ---
    print msg_adopt, msg_adopt_len
    call input_name
    call save_pet           ; WRITE pet.dat
    print msg_hello, msg_hello_len
    jmp .greet
.welcome_back:
    print msg_back, msg_back_len
.greet:
    call print_name
    print msg_excl, msg_excl_len

main_loop:
    print menu, menu_len
    call read_choice
    cmp eax, '1'
    je  opt_feed
    cmp eax, '2'
    je  opt_play
    cmp eax, '3'
    je  opt_sleep
    cmp eax, '4'
    je  opt_status
    cmp eax, '5'
    je  opt_exit
    print msg_badopt, msg_badopt_len
    jmp main_loop

opt_feed:
    call do_feed
    jmp main_loop
opt_play:
    call do_play
    jmp main_loop
opt_sleep:
    call do_sleep
    jmp main_loop
opt_status:
    call do_status
    jmp main_loop
opt_exit:
    call save_pet
    print msg_bye, msg_bye_len
```

```
    exit_program

; ------------------------------------------------------------
; read_choice : reads a line, returns first character in EAX
; (returns '5' on end-of-input so the program saves and exits)
; ------------------------------------------------------------
read_choice:
    mov eax, 3              ; sys_read
    mov ebx, 0              ; stdin
    mov ecx, inbuf
    mov edx, 31
    int 0x80
    test eax, eax
    jle .eof
    movzx eax, byte [inbuf]
    ret
.eof:
    mov eax, '5'
    ret

; ------------------------------------------------------------
; input_name : reads the pet's name (max 15 chars) into pet_name
; an empty name becomes "Pixel"
; ------------------------------------------------------------
input_name:
    mov eax, 3
    mov ebx, 0
    mov ecx, inbuf
    mov edx, 31
    int 0x80
    test eax, eax
    jle .default
    mov esi, inbuf
    mov edi, pet_name
    xor ecx, ecx
.copy:
    movzx eax, byte [esi + ecx]
    cmp eax, 32
    jb  .end                ; Enter / control char -> stop
    mov [edi + ecx], al
    inc ecx
    cmp ecx, 15
    jb  .copy
```

```
.end:
    mov byte [edi + ecx], 0 ; end of name
    test ecx, ecx
    jnz .done
.default:
    mov esi, def_name
    mov edi, pet_name
    mov ecx, 6
    rep movsb
.done:
    ret

; ------------------------------------------------------------
; print_name : prints the pet's name
; ------------------------------------------------------------
print_name:
    pushad
    mov esi, pet_name
    xor edx, edx
.len:
    cmp byte [esi + edx], 0
    je  .go
    inc edx
    cmp edx, 15
    jb  .len
.go:
    mov eax, 4
    mov ebx, 1
    mov ecx, pet_name
    int 0x80
    popad
    ret

; ------------------------------------------------------------
; int_to_str : EAX = number
; returns ESI = start of text in numbuf, ECX = length
; ------------------------------------------------------------
int_to_str:
    mov edi, numbuf + 11
    mov ebx, 10
    xor ecx, ecx
.loop:
    xor edx, edx
```

```
    div ebx
    add dl, '0'
    dec edi
    mov [edi], dl
    inc ecx
    test eax, eax
    jnz .loop
    mov esi, edi
    ret

; ------------------------------------------------------------
; print_int : prints the number in EAX
; ------------------------------------------------------------
print_int:
    pushad
    call int_to_str
    mov edx, ecx
    mov ecx, esi
    mov ebx, 1
    mov eax, 4
    int 0x80
    popad
    ret

; ------------------------------------------------------------
; inc_stat : EBX = address of stat, ECX = amount  (stat max 10)
; ------------------------------------------------------------
inc_stat:
    movzx eax, byte [ebx]
    add eax, ecx
    cmp eax, 10
    jbe .ok
    mov eax, 10
.ok:
    mov [ebx], al
    ret

; ------------------------------------------------------------
; dec_stat : EBX = address of stat, ECX = amount  (stat min 0)
; ------------------------------------------------------------
dec_stat:
    movzx eax, byte [ebx]
    sub eax, ecx
```

```
    jns .ok
    xor eax, eax
.ok:
    mov [ebx], al
    ret

; ------------------------------------------------------------
; draw_bar : EAX = value 0..10, prints  [#####-----]5/10
; ------------------------------------------------------------
draw_bar:
    pushad
    mov esi, eax            ; ESI = value
    mov edi, barbuf + 1
    xor ecx, ecx
.fill:
    mov byte [edi + ecx], '-'
    cmp ecx, esi
    jae .next
    mov byte [edi + ecx], '#'
.next:
    inc ecx
    cmp ecx, 10
    jb  .fill
    print barbuf, 12
    mov eax, esi
    call print_int
    print slash10, 4
    popad
    ret

; ------------------------------------------------------------
; load_pet : READ pet.dat into the pet record
; ------------------------------------------------------------
load_pet:
    mov eax, 5              ; sys_open
    mov ebx, pet_file
    xor ecx, ecx            ; O_RDONLY
    xor edx, edx
    int 0x80
    test eax, eax
    js  .none               ; no saved pet yet
    mov [fd], eax
    mov ebx, eax
```

```
    mov eax, 3              ; sys_read
    mov ecx, pet_name
    mov edx, PET_SIZE
    int 0x80
    cmp eax, PET_SIZE
    jne .skip
    mov dword [has_pet], 1
.skip:
    close_file
.none:
    ret

; ------------------------------------------------------------
; save_pet : WRITE the pet record to pet.dat
; ------------------------------------------------------------
save_pet:
    mov eax, 5              ; sys_open
    mov ebx, pet_file
    mov ecx, 0x241          ; O_WRONLY | O_CREAT | O_TRUNC
    mov edx, 420            ; permissions 0644
    int 0x80
    test eax, eax
    js  .done
    mov [fd], eax
    mov ebx, eax
    mov eax, 4              ; sys_write
    mov ecx, pet_name
    mov edx, PET_SIZE
    int 0x80
    close_file
.done:
    ret

; ------------------------------------------------------------
; do_feed : fullness +3, mood +1
; ------------------------------------------------------------
do_feed:
    cmp byte [pet_full], 10
    jae .full
    raise pet_full, 3
    raise pet_mood, 1
    call save_pet
    call print_name
```

```
    print msg_feed, msg_feed_len
    ret
.full:
    call print_name
    print msg_full, msg_full_len
    ret

; ------------------------------------------------------------
; do_play : mood +3, energy -2, fullness -1 (needs energy >= 2)
; ------------------------------------------------------------
do_play:
    cmp byte [pet_energy], 2
    jb  .tired
    raise pet_mood, 3
    lower pet_energy, 2
    lower pet_full, 1
    call save_pet
    call print_name
    print msg_play, msg_play_len
    ret
.tired:
    call print_name
    print msg_tired, msg_tired_len
    ret

; ------------------------------------------------------------
; do_sleep : energy +5, fullness -1
; ------------------------------------------------------------
do_sleep:
    raise pet_energy, 5
    lower pet_full, 1
    call save_pet
    call print_name
    print msg_sleep, msg_sleep_len
    ret

; ------------------------------------------------------------
; show_face : picks the face using the LOWEST of the 3 stats
;             0-2 = sad, 3-5 = okay, 6-10 = happy
; ------------------------------------------------------------
show_face:
    movzx eax, byte [pet_full]
    movzx ebx, byte [pet_mood]
```

```
    cmp eax, ebx
    jbe .a
    mov eax, ebx
.a:
    movzx ebx, byte [pet_energy]
    cmp eax, ebx
    jbe .b
    mov eax, ebx
.b:
    cmp eax, 3
    jb  .sad
    cmp eax, 6
    jb  .meh
    print face_happy, face_happy_len
    ret
.meh:
    print face_meh, face_meh_len
    ret
.sad:
    print face_sad, face_sad_len
    ret

; ------------------------------------------------------------
; do_status : status card = name, face and three stat bars
; ------------------------------------------------------------
do_status:
    print msg_card, msg_card_len
    call print_name
    print msg_card2, msg_card2_len
    call show_face
    print lbl_full, lbl_len
    movzx eax, byte [pet_full]
    call draw_bar
    print lbl_mood, lbl_len
    movzx eax, byte [pet_mood]
    call draw_bar
    print lbl_energy, lbl_len
    movzx eax, byte [pet_energy]
    call draw_bar
    ret
```